

Engenharia de contexto e RAG para assistente de IA em logística ✓

O Cenário

A NovaTech é uma empresa de médio porte do setor de logística com 1.200 funcionários. Sua operação depende de um conjunto extenso de documentação interna: manuais de procedimento operacional, políticas de compliance, tabelas de SLA por tipo de cliente, regras de cálculo de frete, e normas de segurança de carga.

Hoje, essa documentação está espalhada em três fontes: um SharePoint corporativo com ~800 documentos (PDFs e Word), uma wiki interna no Confluence com ~400 páginas, e uma pasta de rede com planilhas de referência atualizadas mensalmente.

O problema: a equipe de atendimento ao cliente (45 pessoas) gasta em média 12 minutos por chamado buscando informações nessas fontes para responder dúvidas de clientes sobre prazos, regras de frete, políticas de devolução e procedimentos de reclamação. Isso gera atrasos, respostas inconsistentes e frustração tanto dos atendentes quanto dos clientes.

A NovaTech contratou a DB1 para construir um assistente de IA que permita aos atendentes fazer perguntas em linguagem natural e receber respostas fundamentadas na documentação oficial da empresa, com indicação da fonte. O assistente será integrado ao ambiente Microsoft da NovaTech (Teams + SharePoint).

Informações adicionais fornecidas pela NovaTech

- O volume médio é de 320 chamados/dia, dos quais ~60% envolvem consulta a documentação.
- A documentação é atualizada mensalmente por 3 áreas diferentes (Operações, Compliance, Comercial), sem processo unificado de revisão.
- Alguns documentos se contradizem entre versões — a equipe de atendimento hoje resolve isso "perguntando para quem sabe".
- A NovaTech já tem licenças Microsoft 365 E3 e está disposta a provisionar Azure AI Services.
- O projeto tem orçamento para 3 meses de discovery + desenvolvimento + go-live.
- A expectativa da diretoria é reduzir o tempo médio de busca de 12 para menos de 2 minutos por chamado.

Informações técnicas adicionais

"Os PDFs do SharePoint incluem documentos com tabelas complexas (tabelas de frete com 15+ colunas), fluxogramas embutidos como imagens, e alguns documentos escaneados (OCR necessário). A wiki do Confluence tem links internos entre páginas e usa macros customizadas. As planilhas têm fórmulas interdependentes."

Conceito de context engineering aplicado a RAG

"O contexto que o LLM recebe a cada pergunta é limitado pela janela de contexto do modelo. A qualidade da resposta depende de: quais chunks são selecionados (relevância), quantos chunks cabem no contexto (orçamento de atenção), onde ficam posicionados no prompt (informação no meio de contextos longos é 'esquecida' — o efeito 'lost in the middle'), e o que mais está no contexto competindo por atenção (system prompt, histórico de conversa, instruções)."

Prompt:

Desempenhe o papel de um especialista em Engenharia de Contexto e RAG e faça uma análise técnica que cubra:

- Para cada tipo de fonte (PDFs com tabelas, PDFs escaneados, wiki com links, planilhas com fórmulas): qual o desafio para o pipeline de RAG, como isso afeta a qualidade das respostas, e uma estratégia de tratamento.
- Estimativa do tamanho aproximado da base em tokens considerando ~800 documentos PDF (média de 10 páginas cada), ~400 páginas wiki (média de 1.500 palavras cada), e ~50 planilhas. Use a regra prática de ~0.75 palavras por token.
- Análise de orçamento de contexto: dado que o GPT-4o tem 128K tokens de janela e o system prompt + instruções consomem ~2K tokens, quantos chunks de ~500 tokens cabem em cada query? Como isso afeta a estratégia de chunking e retrieval?
- Recomendação de estratégia de chunking justificada pelo tipo de pergunta que o usuário fará e pelo conceito de lost in the middle.

O entregável final é um arquivo markdown completo cobrindo todos os tópicos.

Formato do arquivo:

- Primeira seção:
 - Separe essa seção por tipo de arquivo respondendo as perguntas relevantes em subseções.
 - Faça uso de tabelas se necessário.
- Segunda seção:
 - Estimativa completa da base de dados em tokens
 - Faça uma tabela respondendo além do total, o tamanho de cada tipo de documento e quantos por cento isso é da base total
- Terceira seção:
 - Faça uma tabela com 4 colunas:
 - Modelo de IA
 - Tamanho da janela de contexto
 - Quantos Chunks de ~500 tokens cabem em cada query
 - Como isso afeta a estratégia de chunking e retrieval
 - Antes da tabela forneça a base dos cálculos (Chunks de 500 tokens e system prompt + instruções consumindo 2K tokens)

- Quarta seção:
 - Faça uma tabela contendo:
 - Possível tipo de pergunta do usuário
 - estratégia de Chunking
 - Justificativa pelo tipo de pergunta e conceito do lost in the middle

Pontos importantes para levar em consideração na hora de fazer o documento de análise:

- Quando for realizar cálculos matemáticos, utilizar estratégias de chain-of-thought e ferramentas disponíveis, separando em pequenos passos as contas para evitar alucinações
- ****Não Alucinar****: Caso não conheça informações ou não possua base de dados ou fontes para citar, solicite para mim que irei fornecer

Mostrar menos

Mobilizou estratégia direta para construir documento técnico abrangente >

Documento gerado com 288 linhas e ~3.500 palavras. Aqui está um resumo do que foi coberto em cada seção e as decisões técnicas relevantes:

Seção 1 — Análise por tipo de fonte

Para cada tipo foi documentado o desafio estrutural (não apenas a dificuldade genérica), o impacto direto na qualidade da resposta ao atendente, e uma tabela de estratégia com ferramenta concreta. Os pontos mais críticos por tipo:

- **PDFs com tabelas**: o problema central é a fragmentação de contexto relacional; a solução é serialização contextual linha a linha (`"Tipo Cliente: Gold | Região: SE | SLA: 48h"`) combinada com Azure Document Intelligence.
- **PDFs escaneados**: o ponto mais insidioso é que erros de OCR corrompem o *embedding* — o chunk existe no índice, mas não é recuperado. A solução envolve normalização pós-OCR por dicionário de domínio e score de confiança como metadado.
- **Wiki Confluence**: o problema-raiz são as referências quebradas por links internos; a solução exige exportação via API com rendering completo + resolução de 1 nível de links.
- **Planilhas**: o risco mais alto é indexar fórmulas em vez de valores — `=VLOOKUP(...)` em vez de `"48h"`. A solução é `openpyxl` com `data_only=True`.

Seção 2 — Estimativa de tokens

Foram explicitadas as premissas (300 palavras/pág para PDFs, 3.000 palavras/planilha) com flag de transparência, e todos os cálculos foram detalhados em chain-of-thought.

Total: ~4,2 milhões de tokens, 76,2% provenientes dos PDFs.

Seção 3 — Orçamento de contexto

O overhead de 4.200 tokens (2K system prompt + 200 query + 2K resposta) foi aplicado a 4 modelos. O GPT-4o comporta ~247 chunks teoricamente, mas o limite prático recomendado por *lost in the middle* é **5-15 chunks por query**.

Seção 4 — Chunking por tipo de pergunta

7 tipos de pergunta mapeados com estratégia distinta cada, sempre justificada pelo *lost in the middle* — a lógica de posicionar os chunks mais importantes no início e no final do contexto, nunca no meio.

Duas premissas que precisarão de calibração com dados reais: número de palavras por página dos PDFs (usei 300) e por planilha serializada (usei 3.000). Se tiver uma amostra disponível, consigo refinar os cálculos.



Novatech rag context engineering
Documento · MD

Baixar

